E-GRAMSWARAJ DIGITAL GOVERNANCE PLATFORM FOR VILLAGE PANCHAYAT OPERATIONS

23CS54C – MODERN WEB TECHNOLOGIES

Submitted by

PRIYA M 2403957610422100

In partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



NATIONAL ENGINEERING COLLEGE

(An Autonomous Institution affiliated to Anna University, Chennai)

K.R.NAGAR, KOVILPATTI - 628503

OCTOBER - 2026

NATIONAL ENGINEERING COLLEGE

(An Autonomous Institution affiliated to Anna University, Chennai)

K.R.NAGAR, KOVILPATTI - 628503

BONAFIDE CERTIFICATE

This is to certify that this project report, "E-GramSwaraj – a village management", is the bonafide work of **PRIYA M (2403957610422100)** who carried out the project work under my supervision.

Course Instructor/Guide

Submitted to the 23CS54C – MODERN WEB TECHNOLOGIES Viva-Voce examination held at National Engineering College, K.R.Nagar, Kovilpatti on _____

Internal Examiner **Co Examiner**

TABLE OF CONTENTS

CH NO	TITLE	PAGE NO
1	INTRODUCTION	3
2	OBJECTIVES	4
3	DESCRIPTION	5
4	CLASS DIAGRAM	6
5	TABLE STRUCTURES	7
6	ER DIAGRAM	16
7	TECH STACK	18
8	MODULES	19
9	CONCLUSION	58

CHAPTER 1
INTRODUCTION

In the modern governance ecosystem of rural India, the efficient administration of Panchayati Raj institutions plays a vital role in ensuring sustainable community growth, transparent resource allocation, and accountable public service delivery. Historically, managing village infrastructure projects, local asset registries, financial transactions, citizen welfare requisitions, and Gram Sabha meeting records through paper-based or disconnected ledger systems has been time-consuming, prone to human error, and lacking in public transparency. There is an urgent need for a unified digital platform that automates village panchayat operations under one roof.

E-GramSwaraj is a comprehensive, full-stack digital governance web application designed to digitize and automate the complete lifecycle of village panchayat operations—from resident registration and public complaint processing to multi-year development planning, work execution, asset tracking, financial budget allocation, and dynamic multi-lingual governance. The system was specifically built to cater to the operational requirements of South Thittankulam Village Panchayat (LGD Code: 231907, Kovilpatti Taluk, Tamil Nadu - 628502) while offering multi-tenant architectural readiness for seamless scaling to adjacent villages.

The application is organized into six major functional domains: Master Data & Village Profile Management, which maintains foundational records for village geography, wards, demographics, and resident profiles; Citizen Services & Requisitions, which manages resident self-registration, OTP authentication, and service request workflows; Development Plans & Works Execution, which monitors project lifecycles from proposal to contractor allocation and completion; Financial & Budgetary Control, which enforces strict ledger accounting for income, expenditure, and category-wise budgets; Gram Sabha Meetings & Governance, which records meeting schedules, resolutions, and attendance; and Multi-lingual Localization & Business Intelligence, which powers dynamic 8-language translations via Google Cloud Translation API alongside interactive administrative dashboards.

CHAPTER 2
OBJECTIVES

The objectives of E-GramSwaraj are:

- **Digitize Panchayat Operations –** Replace manual paper registers with an interactive full-stack digital platform for improved operational efficiency, real-time data access, and institutional auditability.
- **Centralize Village Master Data –** Maintain standardized and validated master records for village demographics, ward boundaries, infrastructure assets, and resident citizen catalogs under a single MySQL database source of truth.
- **Automate Development Plan & Works Lifecycle –** Streamline village development from initial proposal to financial sanctioning, contractor assignment, milestone tracking, and completion verification with complete history traceability.
- **Empower Citizens via Requisitions & Complaints –** Provide citizens with direct self-registration, secure OTP verification, and online submission of civic issues (e.g. water supply, street lighting, drainage) with real-time status updates.
- **Ensure Financial Transparency & Budget Integrity –** Record every receipt and expenditure transaction against category-specific annual budgets, automatically computing remaining balances and preventing budget overallocation.
- **Facilitate Digital Gram Sabha Governance –** Schedule panchayat board and Gram Sabha meetings, document formal agendas, log gender-disaggregated public attendance, and store official resolutions in an indexed digital format.
- **Provide Dynamic Multi-lingual Access –** Implement dynamic translations across 8 Indian languages (English, Tamil, Hindi, Telugu, Kannada, Malayalam, Marathi, Bengali) using Google Cloud Translation API backed by a high-speed database cache.
- **Enforce Enterprise Security & Role-Based Access –** Protect sensitive administrative operations using JWT authentication, salted bcrypt password hashing, express rate limiting, Helmet headers, and strict role-based authorization (ADMIN, PANCHAYAT_OFFICER, OVERSEER, WATER_OPERATOR, CITIZEN).

CHAPTER 3
DESCRIPTION

E-GramSwaraj is a web application for modernizing and digitizing Panchayati Raj administrative and public services inside a village panchayat. A visitor lands on a public village information page, registers or signs in, and is directed to an role-tailored dashboard. The top and sidebar navigation expose the core modules: Village Profile, Citizen Services, Works & Development Plans, Asset Management, Financial Ledger & Budgets, Gram Sabha Meetings, Announcements, Notifications, Analytics, and System Settings.

The system is organized into three main functional areas. Village Master Data & Citizen Catalogue covers village geography, population demographics, ward breakdowns, official contacts, facility registries, and resident citizen profiles. Operations & Project Management covers civic grievance lifecycle (Submitted, Verified, In Progress, Resolved, Rejected), civil works tracking from initial proposal to contractor allocation and completion, and GIS asset mapping with maintenance interval logging. Financial Control & Governance covers receipt/expenditure vouchers, category-wise annual budgets, Gram Sabha meeting scheduling, resolutions, attendance recording, and 8-language dynamic localization.

Five roles exist. A Citizen can submit requisitions, track grievance status, view meeting schedules, and access village information. A Panchayat Officer manages village profiles, approves requisitions, schedules Gram Sabha meetings, and monitors works. An Overseer verifies civil engineering progress and asset maintenance. A Water Operator manages water supply schemes and civic utility assets. An Admin possesses full system privilege to manage user roles, system settings, and security policies. Route protection is enforced in the browser by a protected-route wrapper and on the server by JWT authentication and role-based authorization middleware.

E-GramSwaraj runs in two cooperating modes. The React client maintains a seeded, working dataset in browser local storage and offline cache, allowing screens to render essential data when offline. When the Express REST API backend answers its health check, the application seamlessly communicates with Node.js and Sequelize ORM over MySQL 8.

CHAPTER 4
CLASS DIAGRAM

The class diagram shows the persistent entities of E-GramSwaraj+ together with the controller/service operations that act on them. Every Sequelize ORM model is represented as a class with its attributes, validations, and controller methods that read or modify it. Relationships are logical and relational foreign keys, for example User.villageId refers to Village.id and Work.planId refers to DevelopmentPlan.id.

[PASTE SCREENSHOT HERE: Fig 4.1 Class diagram of E-GramSwaraj+]

(Leave space provided box / Replace this box with your image screenshot)

Fig 4.1 Class diagram of E-GramSwaraj+

CHAPTER 5
TABLE STRUCTURES

E-GramSwaraj+ uses MySQL 8, a relational database, with fifteen Sequelize ORM tables. Each table below is the schema of one model: field name, data type, whether it is required (Null = NO), key type (PRI primary key, UNI unique, MUL foreign key) and the default value. Every document receives an automatic _id of type UUID. Sequelize adds createdAt and updatedAt timestamps automatically.

5.1 VILLAGES (villages)

Stores foundational reference data for South Thittankulam Village Panchayat, including geography, population statistics, household counts, ward breakdowns, and official contacts.

Field	Type	Null	Key	Default
id	UUID	NO	PRI	UUIDV4
name	String (varchar 255)	NO		null
villageCode	String (varchar 255)	NO	UNI	STK
panchayatName	String (varchar 255)	YES		null
taluk	String (varchar 255)	NO		Kovilpatti
district	String (varchar 255)	YES		Thoothukkudi
state	String (varchar 255)	NO		Tamil Nadu
pincode	String (varchar 255)	NO		628502
population	Integer	YES		14317
households	Integer	YES		5386
wards	Integer	YES		9
area	Decimal (14,2)	YES		4.0
latitude	Float	YES		9.1720
longitude	Float	YES		77.8810
lgdCode	String (varchar 255)	YES		'231907'

Field	Type	Null	Key	Default
status	enum(ACTIVE, INACTIVE)	NO		ACTIVE
createdAt	Date	NO		timestamps

Table 5.1 Structure of the villages collection

5.2 USERS (users)

Stores every registered administrative officer and citizen account. Passwords are stored only as salted bcrypt hashes (cost factor 12), and role separates citizens from administrative officers.

Field	Type	Null	Key	Default
_id	UUID	NO	PRI	UUIDV4
name	String (varchar 255)	NO		null
email	String (varchar 255)	NO	UNI	null
password	String (bcrypt hash)	NO		null
role	enum(ADMIN, OFFICER, OVERSEER, WATER_OPERATOR, CITIZEN)	NO		CITIZEN
villageId	UUID	NO	MUL	FK -> villages
phone	String (varchar 255)	YES	UNI	null
designation	String (varchar 255)	YES		null
language	enum(en, ta, hi, te, kn, ml, mr, bn)	YES		en
status	enum(ACTIVE, INACTIVE)	NO		ACTIVE
lastLoginAt	Date	YES		null
createdAt / updatedAt	Date	NO		timestamps

Table 5.2 Structure of the users collection

5.3 CITIZENS (citizens)

Holds resident citizen profiles linked to specific village wards, phone numbers, and optional user portal account logins.

Field	Type	Null	Key	Default
_id	UUID	NO	PRI	UUIDV4
villageId	UUID	NO	MUL	FK -> villages
name	String (varchar 255)	NO		null
phone	String (varchar 255)	YES		null
email	String (varchar 255)	YES		null
address	String (varchar 255)	YES		null
ward	String (varchar 255)	YES		Ward 1
gender	enum(MALE, FEMALE, OTHER, '')	YES		''
userId	UUID	YES	MUL	FK -> users
status	enum(ACTIVE, INACTIVE)	NO		ACTIVE
createdAt / updatedAt	Date	NO		timestamps

Table 5.3 Structure of the citizens collection

5.4 DEVELOPMENT PLANS (developmentPlans)

Stores multi-year Gram Panchayat Development Plans (GPDP) with categories, priority levels, estimated costs, and target completion dates.

Field	Type	Null	Key	Default
_id	UUID	NO	PRI	UUIDV4
villageId	UUID	NO	MUL	FK -> villages
title	String (varchar 255)	NO		null
description	Text	YES		null
category	enum(Infrastructure, Water, Sanitation, Roads, Education, Health)	NO		Infrastructure

Field	Type	Null	Key	Default
priority	enum(LOW, MEDIUM, HIGH, URGENT)	NO		MEDIUM
estimatedCost	Decimal (14,2)	YES		0.00
status	enum(PROPOSED, APPROVED, IN_PROGRESS, COMPLETED, REJECTED)	NO		PROPOSED
progress	Float	YES		0
financialYear	String (varchar 255)	YES		2026-2027
createdBy	UUID	YES	MUL	FK -> users
createdAt / updatedAt	Date	NO		timestamps

Table 5.4 Structure of the developmentPlans collection

5.5 WORKS (works)

Monitors active civil engineering works, tracking activity codes, scheme allocations, estimated vs approved costs, assigned contractors, and completion percentages.

Field	Type	Null	Key	Default
_id	UUID	NO	PRI	UUIDV4
villageId	UUID	NO	MUL	FK -> villages
name	String (varchar 255)	NO		null
activityCode	String (varchar 255)	YES		null
workType	String (varchar 255)	YES		New/Fresh
category	enum(Infrastructure, Water, Sanitation, Roads)	YES		null
location	String (varchar 255)	YES		null
estimatedCost	Decimal (14,2)	YES		0.00
approvedAmount	Decimal (14,2)	YES		0.00
spentAmount	Decimal (14,2)	YES		0.00

Field	Type	Null	Key	Default
contractor	String (varchar 255)	YES		null
status	enum(PLANNED, APPROVED, IN_PROGRESS, COMPLETED, SUSPENDED)	NO		PLANNED
progress	Float	YES		0
actualCompletionDate	Date	YES		null
createdAt / updatedAt	Date	NO		timestamps

Table 5.5 Structure of the works collection

5.6 ASSETS (assets)

Catalogs public physical assets (water tanks, hand pumps, street lights, community halls, roads) with GIS latitude/longitude coordinates and maintenance interval logs.

Field	Type	Null	Key	Default
_id	UUID	NO	PRI	UUIDV4
villageId	UUID	NO	MUL	FK -> villages
name	String (varchar 255)	NO		null
assetCode	String (varchar 255)	YES		null
type	enum(WATER_TANK, STREET_LIGHT, RO_PLANT, BUILDING, ROAD, DRAINAGE)	NO		null
ward	String (varchar 255)	YES		null
latitude	Float (-90 to 90)	YES		null
longitude	Float (-180 to 180)	YES		null
condition	enum(EXCELLENT, GOOD, POOR, NON_FUNCTIONAL)	NO		GOOD
estimatedValue	Decimal (14,2)	YES		0.00

Field	Type	Null	Key	Default
maintenanceIntervalDays	Integer	YES		180
status	enum(ACTIVE, INACTIVE, UNDER_MAINTENANCE)	NO		ACTIVE
createdAt / updatedAt	Date	NO		timestamps

Table 5.6 Structure of the assets collection

5.7 FINANCIAL TRANSACTIONS (financialTransactions)

Records public financial income receipts and expenditure payments linked to voucher references and financial years.

Field	Type	Null	Key	Default
_id	UUID	NO	PRI	UUIDV4
villageId	UUID	NO	MUL	FK -> villages
type	enum(RECEIPT, EXPENDITURE)	NO		null
category	String (varchar 255)	NO		null
description	Text	YES		null
amount	Decimal (14,2)	NO		null
referenceNumber	String (varchar 255)	YES		null
date	Date	NO		CURRENT_TIMESTAMP
financialYear	String (varchar 255)	YES	MUL	Auto-computed
status	enum(COMPLETED, PENDING, CANCELLED)	NO		COMPLETED
createdAt / updatedAt	Date	NO		timestamps

Table 5.7 Structure of the financialTransactions collection

5.8 BUDGETS (budgets)

Stores annual category budgets with allocated, utilized, and auto-computed remaining funds.

Field	Type	Null	Key	Default
_id	UUID	NO	PRI	UUIDV4
villageId	UUID	NO	MUL	FK -> villages
financialYear	String (varchar 255)	NO	MUL	null
category	String (varchar 255)	NO	MUL	null
allocatedAmount	Decimal (14,2)	NO		0.00
utilizedAmount	Decimal (14,2)	YES		0.00
remainingAmount	Decimal (14,2)	YES		Auto-calculated
source	String (varchar 255)	YES		State Grant / Tax
createdAt / updatedAt	Date	NO		timestamps

Table 5.8 Structure of the budgets collection

5.9 MEETINGS (meetings)

Schedules Gram Sabha and Panchayat Board assembly meetings, documenting agendas, resolutions, and public attendance counts.

Field	Type	Null	Key	Default
_id	UUID	NO	PRI	UUIDV4
villageId	UUID	NO	MUL	FK -> villages
title	String (varchar 255)	NO		null
type	enum(GRAM_SABHA, PANCHAYAT_BOARD, WARD_SABHA, STANDING_COMMITTEE)	NO		GRAM_SABHA
date	Date	NO		null
location	String (varchar 255)	YES		Panchayat Office

Field	Type	Null	Key	Default
agenda	JSON Array	YES		[]
minutes	Text	YES		null
decisions	JSON Array	YES		[]
attendance	JSON Object (total, male, female, officials)	YES		{}
status	enum(SCHEDULED, IN_PROGRESS, COMPLETED, CANCELLED)	NO		SCHEDULED
createdAt / updatedAt	Date	NO		timestamps

Table 5.9 Structure of the meetings collection

5.10 ANNOUNCEMENTS (announcements)

Stores public notices, policy advisories, and village news bulletins.

Field	Type	Null	Key	Default
_id	UUID	NO	PRI	UUIDV4
villageId	UUID	NO	MUL	FK -> villages
title	String (varchar 255)	NO		null
description	Text	NO		null
category	enum(PUBLIC_INFORMATION, SCHEME_NOTICE, EMERGENCY_ALERT)	NO		PUBLIC_INFORMATION
publishDate	Date	NO		CURRENT_TIMESTAMP
status	enum(PUBLISHED, DRAFT, ARCHIVED)	NO		PUBLISHED
createdAt / updatedAt	Date	NO		timestamps

Table 5.10 Structure of the announcements collection

5.11 NOTIFICATIONS (notifications)

In-app alert notifications generated upon requisition status updates, meeting schedules, or system events.

Field	Type	Null	Key	Default
_id	UUID	NO	PRI	UUIDV4
villageId	UUID	NO	MUL	FK -> villages
userId	UUID	NO	MUL	FK -> users
title	String (varchar 255)	NO		null
message	Text	YES		null
type	enum(SYSTEM, GRIEVANCE, WORK, MEETING)	NO		SYSTEM
read	Boolean	NO		false
createdAt / updatedAt	Date	NO		timestamps

Table 5.11 Structure of the notifications collection

5.12 GRIEVANCES (grievances)

Stores public civic complaint submissions, issue categories, resolution stages, and audit history logs.

Field	Type	Null	Key	Default
_id	UUID	NO	PRI	UUIDV4
villageId	UUID	NO	MUL	FK -> villages
referenceNumber	String (varchar 255)	NO	UNI	Auto-generated
citizenId	UUID	NO	MUL	FK -> citizens
category	enum(Water Supply, Street Lighting, Drainage, Roads, Garbage)	NO		null
subject	String (varchar 255)	NO		null
description	Text	NO		null

Field	Type	Null	Key	Default
ward	String (varchar 255)	YES		null
priority	enum(LOW, MEDIUM, HIGH, URGENT)	NO		MEDIUM
status	enum(SUBMITTED, ACKNOWLEDGED, IN_PROGRESS, RESOLVED, REJECTED)	NO		SUBMITTED
resolution	Text	YES		null
createdAt / updatedAt	Date	NO		timestamps

Table 5.12 Structure of the grievances collection

5.13 SERVICE REQUESTS (serviceRequests)

Handles citizen applications for official certificates, public service permits, and administrative utility connections.

Field	Type	Null	Key	Default
_id	UUID	NO	PRI	UUIDV4
villageId	UUID	NO	MUL	FK -> villages
applicationNumber	String (varchar 255)	NO	UNI	SR-XXXXXXXX
serviceId	UUID	NO	MUL	FK -> services
applicantName	String (varchar 255)	NO		null
applicantPhone	String (varchar 255)	NO		null
ward	String (varchar 255)	YES		null
status	enum(SUBMITTED, VERIFIED, IN_PROGRESS, APPROVED, REJECTED, COMPLETED)	NO		SUBMITTED

Field	Type	Null	Key	Default
assignedOfficer	UUID	YES	MUL	FK -> users
slaDays	Integer	YES		7
createdAt / updatedAt	Date	NO		timestamps

Table 5.13 Structure of the serviceRequests collection

5.14 TRANSLATION CACHE (translationCache)

Stores SHA-256 hashed UI source text and dynamic Google Cloud Translation API outputs across 8 Indian languages.

Field	Type	Null	Key	Default
_id	UUID	NO	PRI	UUIDV4
sourceHash	String (varchar 64)	NO	UNI	SHA-256 Hash
sourceText	Text	NO		null
sourceLanguage	String (varchar 8)	NO		en
targetLanguage	String (varchar 8)	NO		ta / hi / te / etc.
translatedText	Text	NO		null
createdAt / updatedAt	Date	NO		timestamps

Table 5.14 Structure of the translationCache collection

CHAPTER 6
ER DIAGRAM

The entity-relationship diagram shows how the collections relate to one another. MySQL enforces relational foreign key constraints across tables (`villageId`, `userId`, `planId`, `workId`, `serviceId`). The villages collection is the central hub of the model: a village contains users, citizens, development plans, works, assets, financial transactions, budgets, meetings, and service requests.

ENTITY RELATIONSHIP DIAGRAM

📻 **[PASTE SCREENSHOT HERE: Fig 6.1 Entity relationship diagram of E-GramSwaraj+]**

(Leave space provided box / Replace this box with your image screenshot)

Fig 6.1 Entity relationship diagram of E-GramSwaraj+

RELATIONSHIP SUMMARY

Parent	Child	Cardinality	Reference field
villages	users	1 : N	users.villageId → villages.id
villages	citizens	1 : N	citizens.villageId → villages.id
villages	developmentPlans	1 : N	developmentPlans.villageId → villages.id
villages	works	1 : N	works.villageId → villages.id
villages	assets	1 : N	assets.villageId → villages.id
villages	financialTransactions	1 : N	financialTransactions.villageId → villages.id
villages	budgets	1 : N	budgets.villageId → villages.id
villages	meetings	1 : N	meetings.villageId → villages.id
villages	announcements	1 : N	announcements.villageId → villages.id
users	grievances	1 : N	grievances.citizenId → users.id
services	serviceRequests	1 : N	serviceRequests.serviceId → services.id
users	otpVerifications	1 : N	otpVerifications.userId → users.id

Table 6.1 Relationship Summary of E-GramSwaraj+ Collections

CHAPTER 7
TECH STACK

7.1 TECH STACK

Layer	Technology	Purpose
Frontend UI	React 18 (Vite)	Single-page application with dynamic UI rendering and fast module reloading.
Styling	Tailwind CSS	Utility-first responsive styling with consistent design system tokens.
Routing	React Router v6	Public, citizen, and officer routes guarded by role-based protected wrappers.
Charts & Maps	Recharts, Leaflet, OpenStreetMap	Interactive financial analytics charts and GIS asset spatial mapping.
Backend Server	Node.js with Express v4	Asynchronous, event-driven RESTful API server.
Database & ORM	MySQL 8 + Sequelize ORM v6	Relational database storage with schema migrations, validations, and hooks.
Authentication	JSON Web Tokens (jsonwebtoken)	Stateless, signed 7-day bearer token authentication.
Password Security	bcryptjs	Salted password hashing with cost factor 12.
HTTP Hardening	Helmet, CORS, Express Rate Limit	HTTP header protection, origin control, and rate limiting.
Localization	Google Cloud Translation API + Cache	Dynamic real-time translation across 8 Indian languages with SHA-256 caching.

Table 7.1 Overview of E-GramSwaraj+ Technology Stack

7.2 ARCHITECTURE OVERVIEW

Layer	Technology Component	Responsibility in E-GramSwaraj+
Model	Sequelize ORM + MySQL 8	Defines relational schemas, data validations, DB queries, and lifecycle hooks.
View	React 18 UI + Tailwind CSS	Renders interactive interfaces, manages client state, displays charts & maps.
Controller	Express.js Route Controllers	Executes business logic, validates API payloads, coordinates Model & View.

Table 7.2 Architecture Layer Overview

CHAPTER 8
MODULES

8.2 Folder Structure

The project is a single repository with two independent applications: the Express API in backend and the React client in frontend. Figure 8.1 shows the backend directory structure, and Figure 8.2 shows the frontend directory structure.

Backend Folder

The backend repository contains API route handlers, Sequelize models, security middleware, seeders, and translation services.

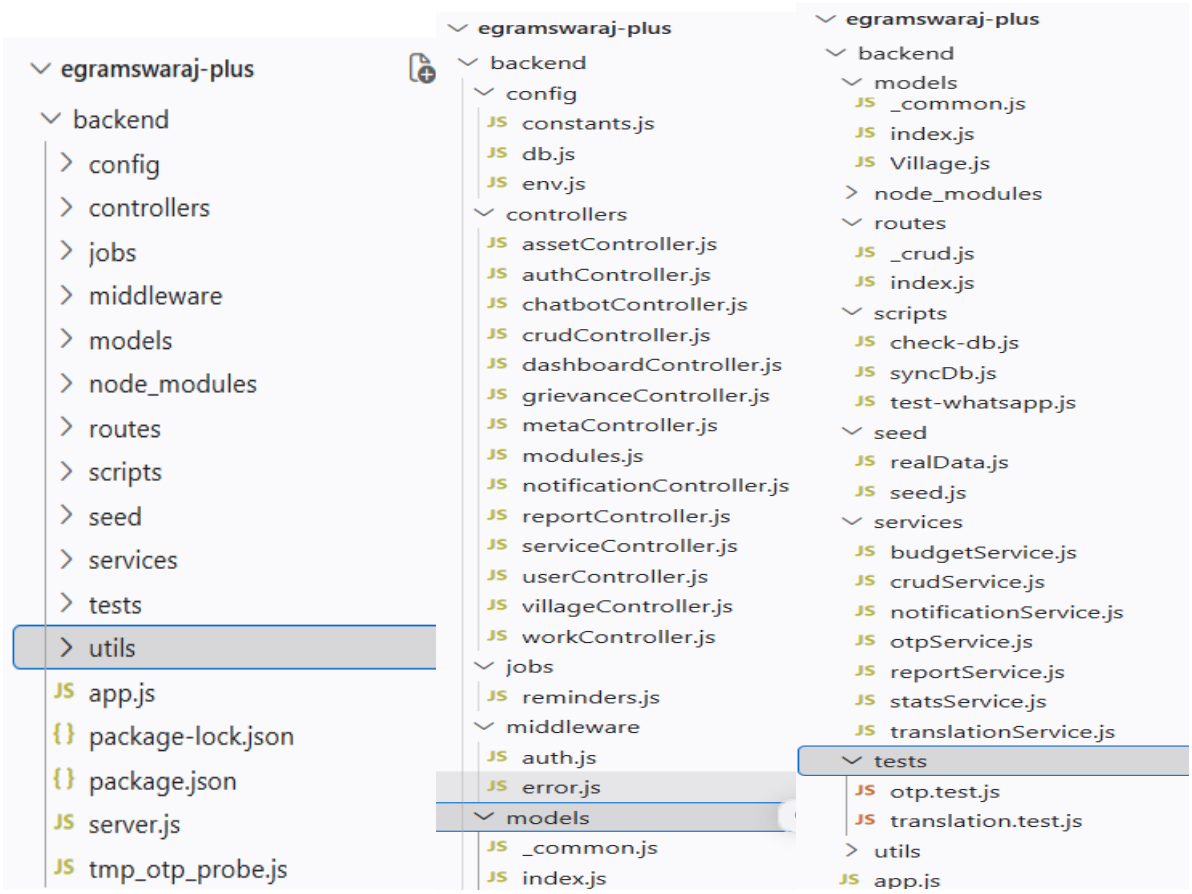


Fig 8.1 Structure of backend folder

Frontend Folder

The frontend repository contains React components, UI page views, custom context providers, and Axios service integrations.

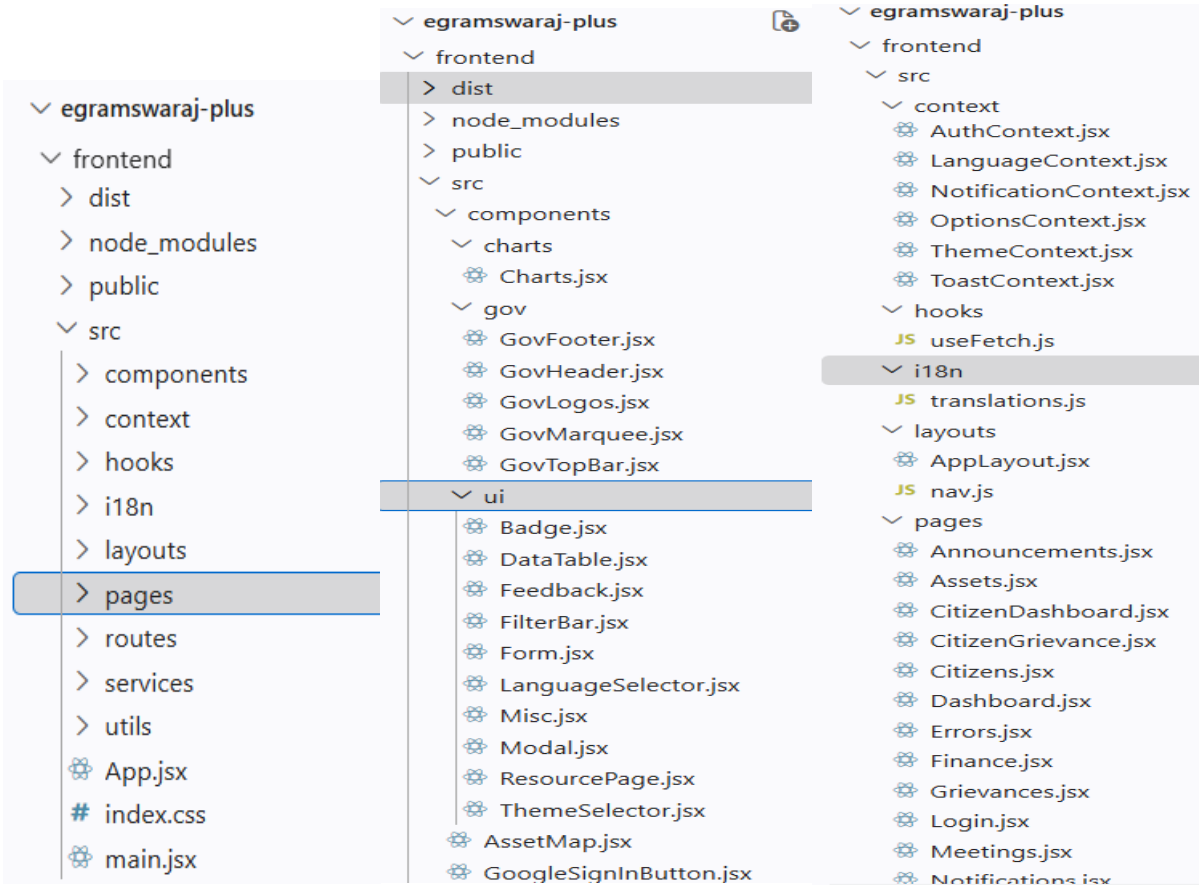


Fig 8.2 Structure of frontend folder

8.3 USER AUTHENTICATION & ACCESS CONTROL

Purpose & Scope

Provides account creation, sign-in, OTP validation, and role-based access control (RBAC) across administrative and citizen user accounts. It identifies every user with a signed JSON Web Token (JWT), separates citizens from officers, overseers, and administrators, and prevents unauthorized access to administrative functions.

Functional Requirements

What the module must do:

- A citizen can register with name, e-mail, phone, ward address, and password (minimum eight characters) with automated OTP verification.

- A user can log in with e-mail or phone number and password, receiving a signed 7-day JWT Bearer token.
- Role-based authorization enforces 5 distinct roles: ADMIN, PANCHAYAT_OFFICER, OVERSEER, WATER_OPERATOR, and CITIZEN.
- An administrator can view user accounts, update designations, or suspend inactive accounts from the administration panel.

Non-Functional Requirements

How well the module must do it:

- **Security:** Passwords are stored only as salted bcrypt hashes (cost factor 12) and are never returned by the API.
- **Token lifetime:** Server JWT tokens expire after 7 days to balance administrative security with usability.
- **Rate limiting:** Login attempts are rate-limited to 15 attempts per 15-minute window per IP.
- **Consistency:** E-mail addresses and phone numbers are trimmed, lower-cased, and unique.

API Design

Method	Endpoint	Purpose / contract
POST	/api/users/login	Body: email/phone, password. Authenticates credentials and returns signed JWT.
POST	/api/users/register	Body: name, email, phone, password, ward. Registers citizen profile & triggers OTP.
POST	/api/users/verify-otp	Body: userIdentifier, otp. Verifies OTP and activates user account.
GET	/api/users/me	Header: Bearer token. Returns authenticated user profile.

DB Design

Collection	Keys & indexes	Key fields
users	_id, unique email, unique phone	name, email, password (hash), role, villageId, phone, status
otpVerifications	_id, userIdentifier, purpose	userIdentifier, otpHash, purpose, expiresAt, verified

- The unique indexes on email and phone prevent duplicate registrations.
- The role field is an enum enforcing RBAC authorization across routes.

Screenshots

The screens below show the user login interface, citizen registration page, and OTP verification dialog.

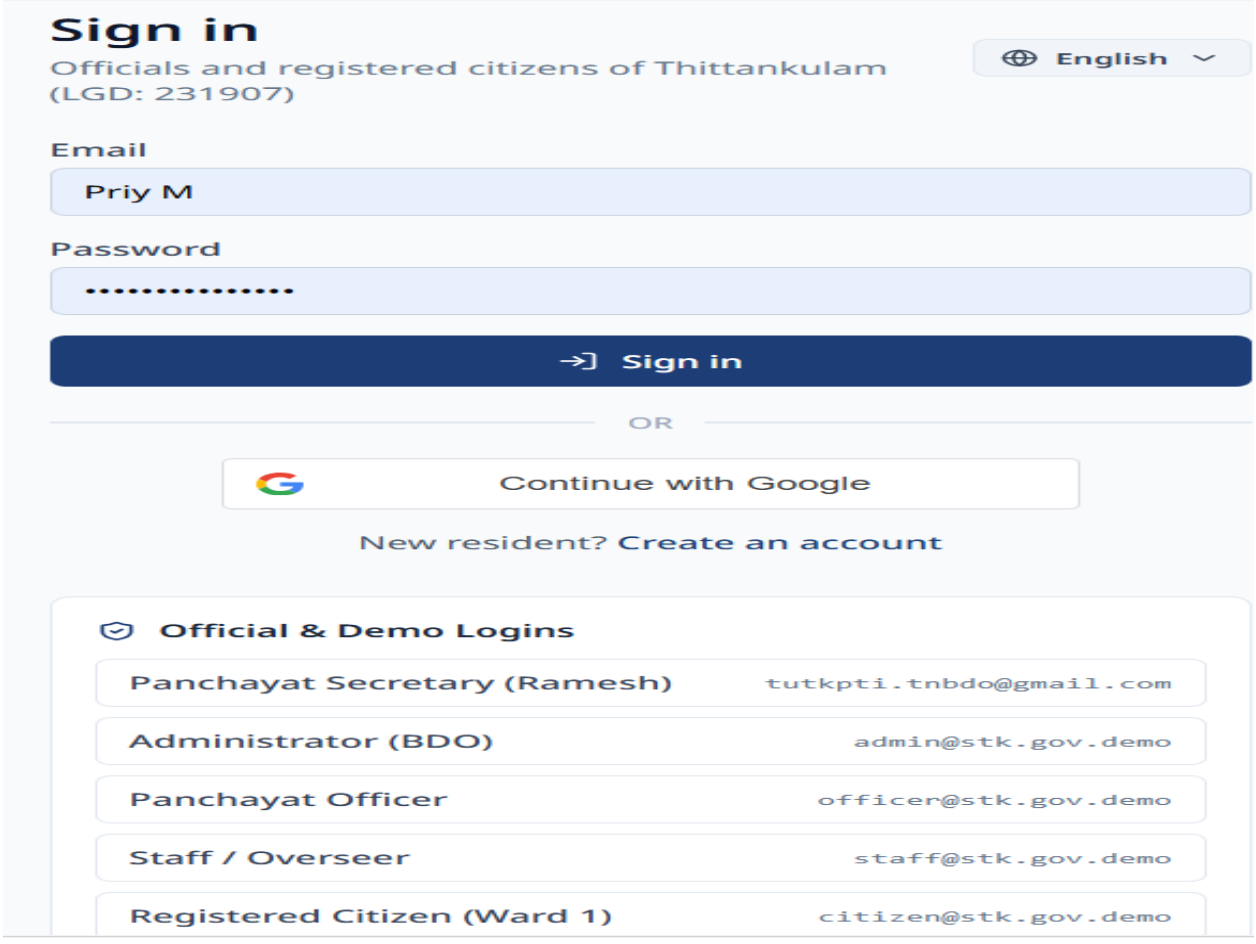


Fig 8.3 User login screen with role-based login options

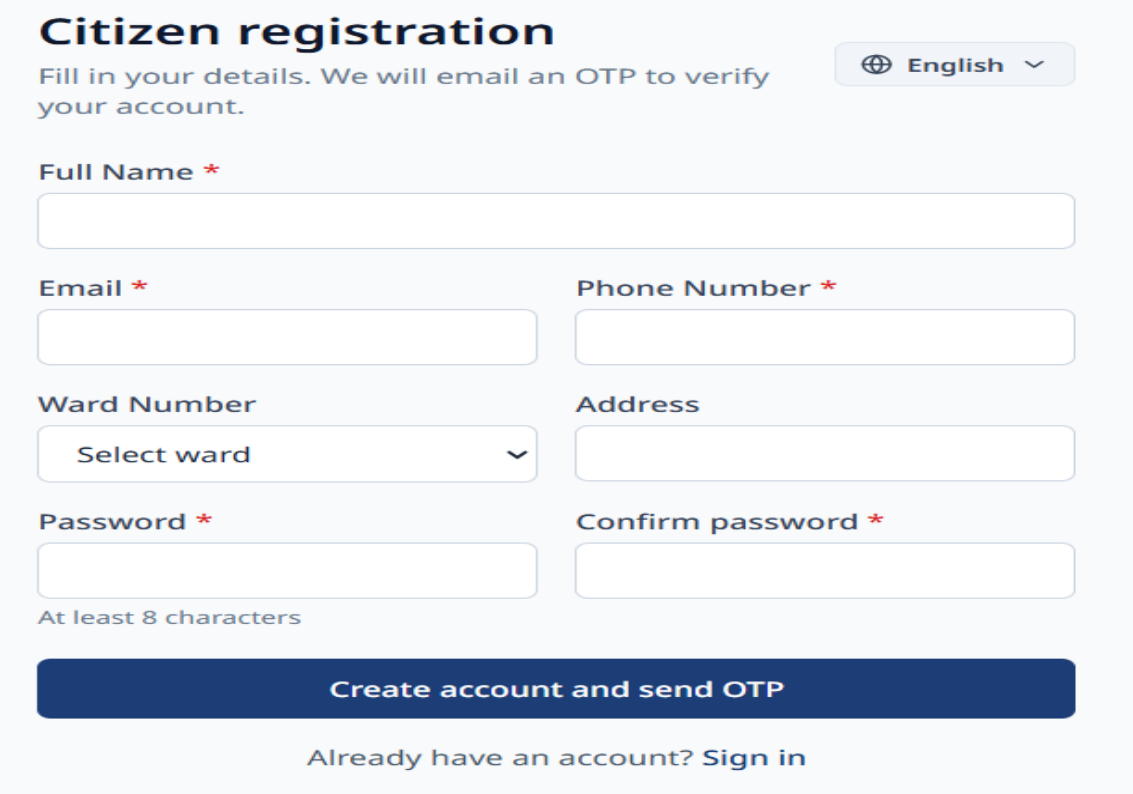


Fig 8.4 Citizen registration and profile creation screen

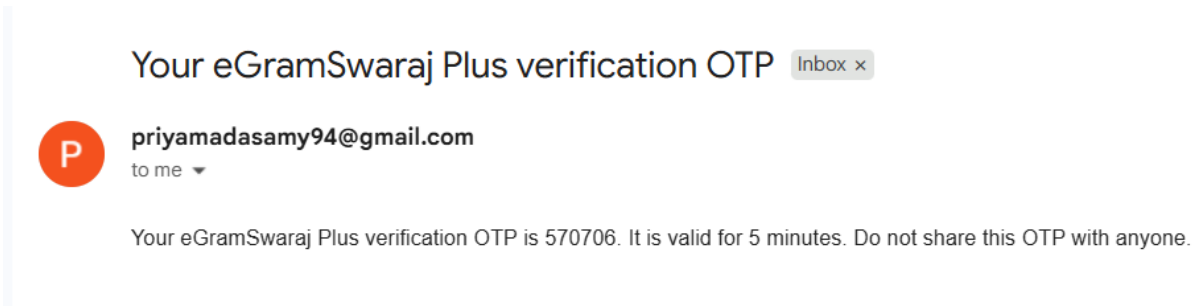


Fig 8.5 OTP Verification dialog for citizen onboarding

8.4 VILLAGE PROFILE & MASTER DATA MANAGEMENT

Purpose & Scope

Centralizes foundational reference data for South Thittankulam Village Panchayat (LGD Code: 231907), including geography, population statistics, household counts, ward breakdowns, official contact directories, and village facility registries.

Functional Requirements

What the module must do:

- Maintain official village details: name, code ('STK'), LGD code (231907), block, taluk, district, state, and pincode.
- Record population, household counts, ward divisions, geographic boundary area, and local public amenities.
- Provide a facility registry detailing panchayat office, primary health center, schools, community halls, and water overhead tanks.

Non-Functional Requirements

How well the module must do it:

- **Data Integrity:** Enforce unique key constraints on villageCode and LGD code.
- **Read Optimization:** Village profile data is cached in client memory on startup to optimize load speed.

API Design

Method	Endpoint	Purpose / contract
GET	/api/villages	Lists all village panchayats in the system.
GET	/api/villages/:code	Fetches complete master profile & demographics for a village code.
PUT	/api/villages/:id	Admin/Officer only. Updates village statistics, contact officers, or amenities.

DB Design

Collection	Keys & indexes	Key fields
villages	_id, unique villageCode, lgdCode	name, villageCode, taluk, district, state, population, lgdCode
citizens	_id, villageId, userId	villageId, name, phone, email, address, ward, gender, status

Screenshots

The screens below show the village profile interface and demographic master data views.

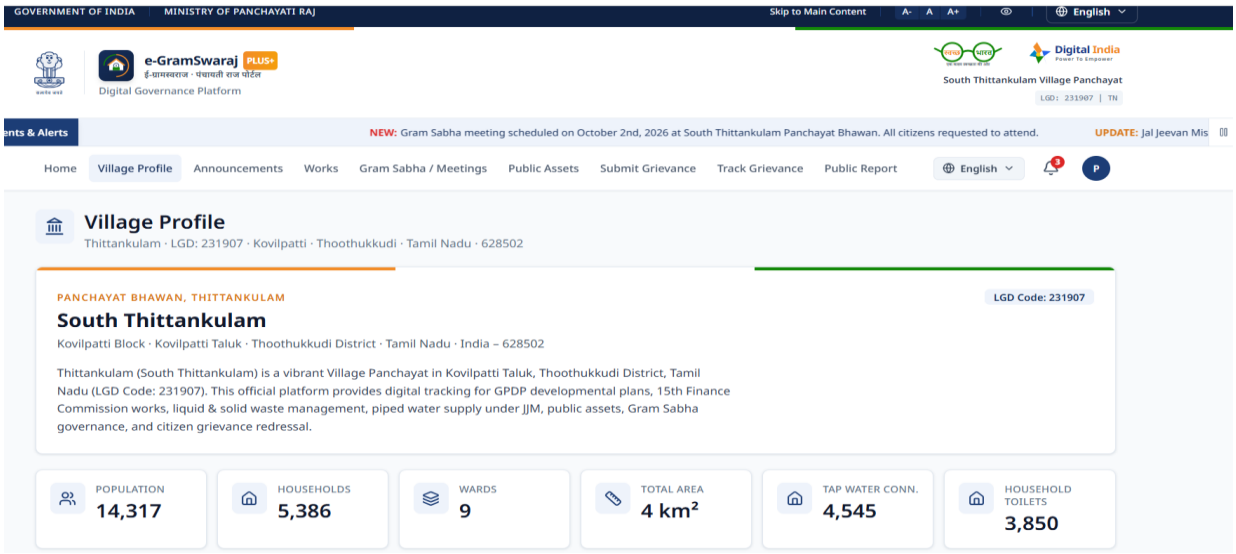


Fig 8.6 Village profile master data and demographics interface

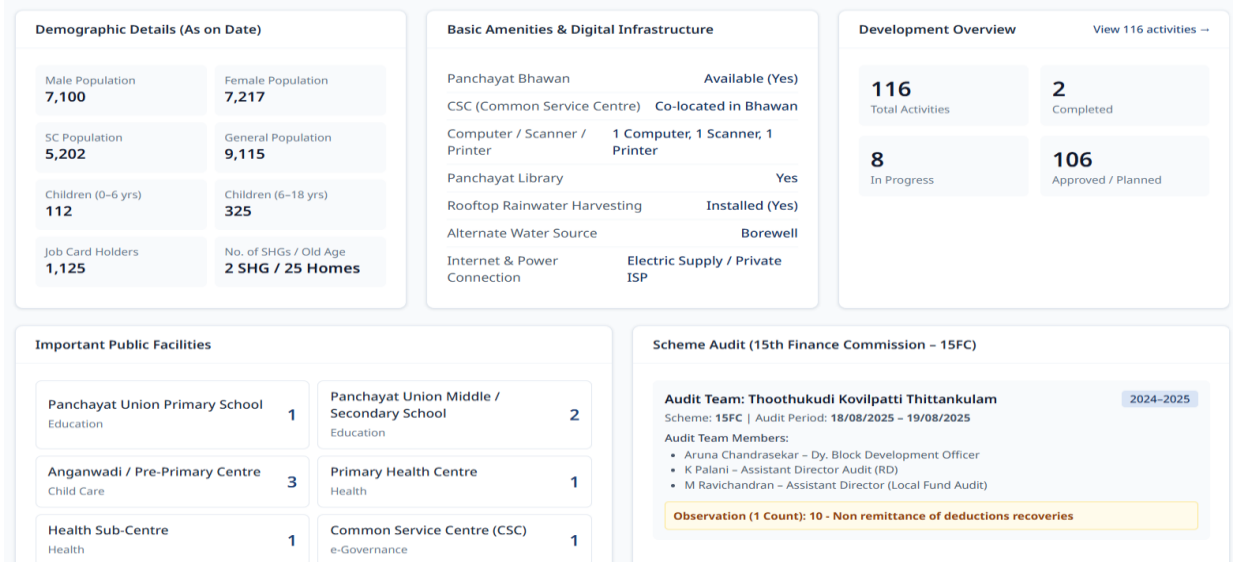


Fig 8.7 Village facility and public infrastructure directory

8.5 CITIZEN SERVICES & REQUISITIONS / GRIEVANCES MANAGEMENT

Purpose & Scope

Empowers citizens to submit civic requisitions and public complaints (water supply, street lighting, drainage, road repairs, garbage removal) online, track resolution workflows, and receive status updates.

Functional Requirements

What the module must do:

- A citizen can submit a grievance with category, subject, description, ward location, and priority level.
- Generates a unique tracking reference number (e.g. GRV-202610-XXXX) upon submission.
- Panchayat officers can assign grievances to field staff and update resolution status (Submitted, Acknowledged, In Progress, Resolved).

Non-Functional Requirements

How well the module must do it:

- **Traceability:** All status transitions are appended to an immutable audit history with timestamps.
- **Responsiveness:** Citizens can view real-time status changes from their personal portal.

API Design

Method	Endpoint	Purpose / contract
GET	/api/grievances	Lists grievances filtered by status, category, or citizen ID.
POST	/api/grievances	Creates a new civic complaint requisition and returns reference number.
PUT	/api/grievances/:id/status	Officer only. Updates grievance resolution status and appends remarks.

DB Design

Collection	Keys & indexes	Key fields
grievances	_id, unique referenceNumber, citizenId	referenceNumber, citizenId, category, subject, ward, status, history
serviceRequests	_id, unique applicationNumber	applicationNumber, serviceId, applicantName, status, assignedOfficer

Screenshots

The screens below show the grievance submission portal and issue tracking workflow.

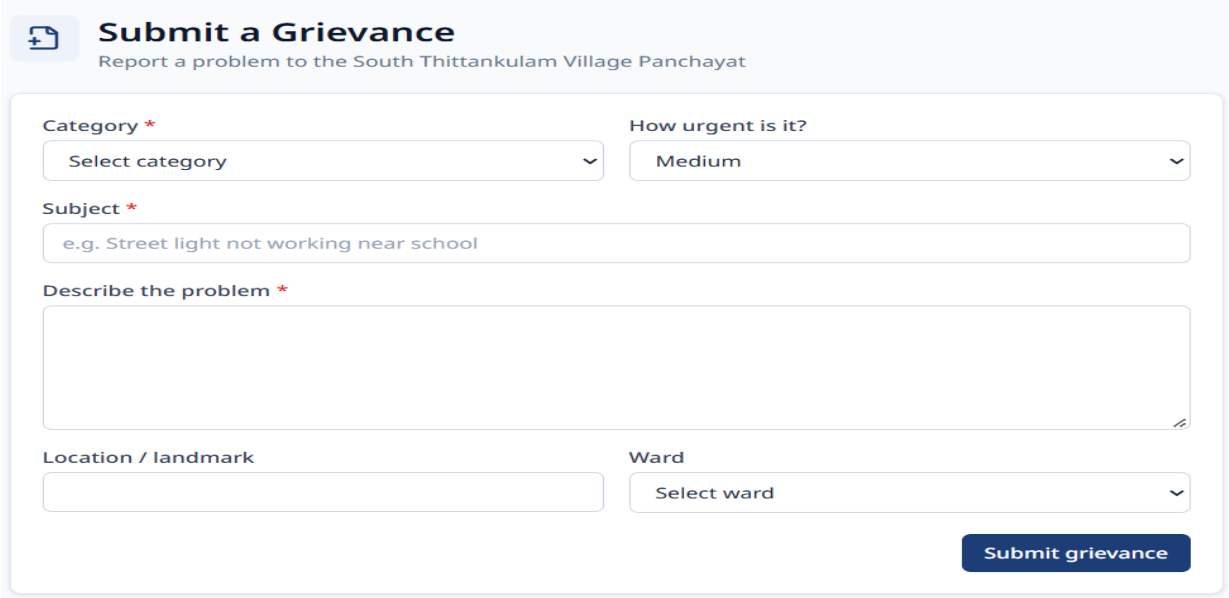


Fig 8.8 Citizen civic grievance submission portal

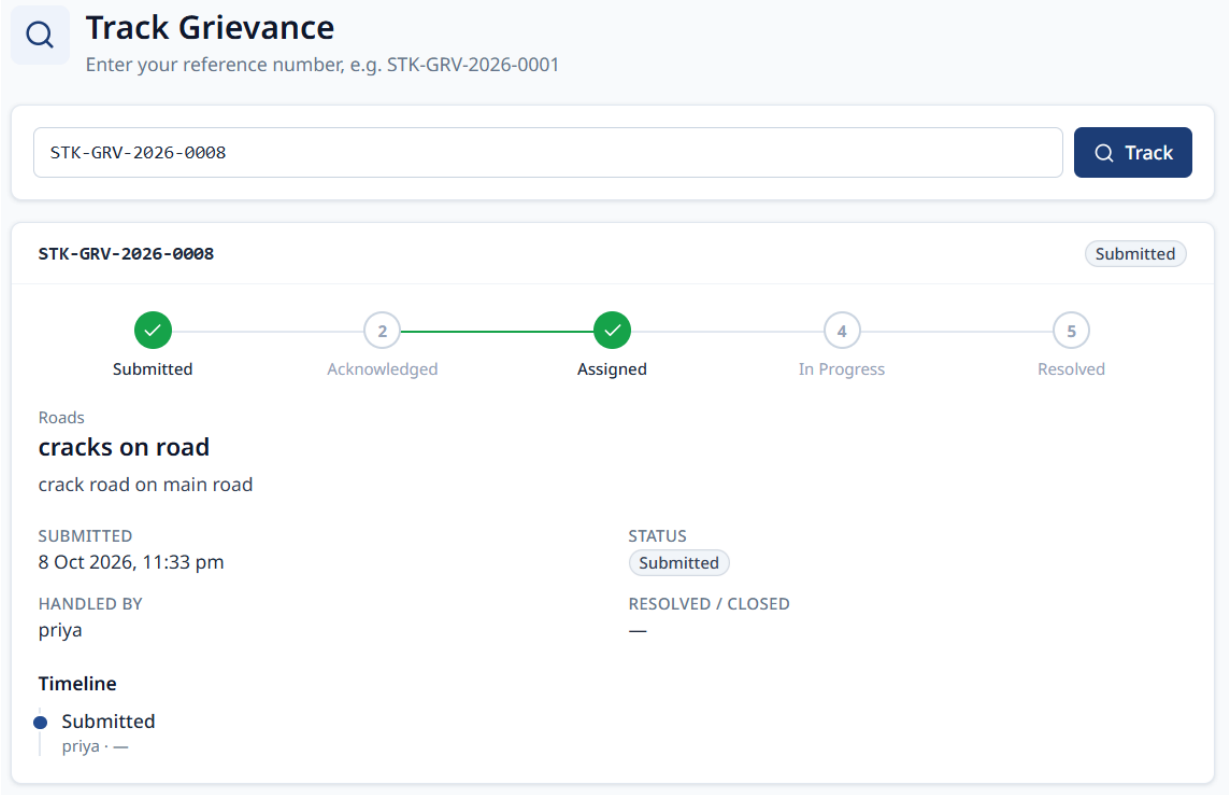


Fig 8.9 Grievance status tracking and resolution workflow

8.6 DEVELOPMENT PLANS & WORKS EXECUTION

Purpose & Scope

Orchestrates Gram Panchayat Development Plans (GPDP) and active civil infrastructure works from initial proposal to financial sanctioning, contractor assignment, and milestone completion.

Functional Requirements

What the module must do:

- Create multi-year development plans with categories (Infrastructure, Water, Sanitation, Roads), priority, and estimated cost.
- Track active civil works with activity codes, location coordinates, scheme allocations, and contractor assignments.
- Update physical progress (0-100%); auto-mark status COMPLETED when progress reaches 100%.

Non-Functional Requirements

How well the module must do it:

- **Precision:** All cost estimates and expenditure amounts are stored as DECIMAL(14,2).
- **Automated Hook:** Sequelize beforeSave hook auto-updates actualCompletionDate upon 100% progress.

API Design

Method	Endpoint	Purpose / contract
GET	/api/works	Lists civil works filtered by status, category, or financial year.
POST	/api/works	Officer only. Creates a new civil work under a sanctioned development plan.
PUT	/api/works/:id	Overseer/Officer. Updates physical progress percentage, spent amount, or status.

DB Design

Collection	Keys & indexes	Key fields
developmentPlans	_id, villageId, status	title, category, priority, estimatedCost, status, financialYear
works	_id, villageId, status	name, activityCode, category, location, estimatedCost, spentAmount, contractor, progress, status

Screenshots

The screens below show the development plan listing and civil works progress tracking.

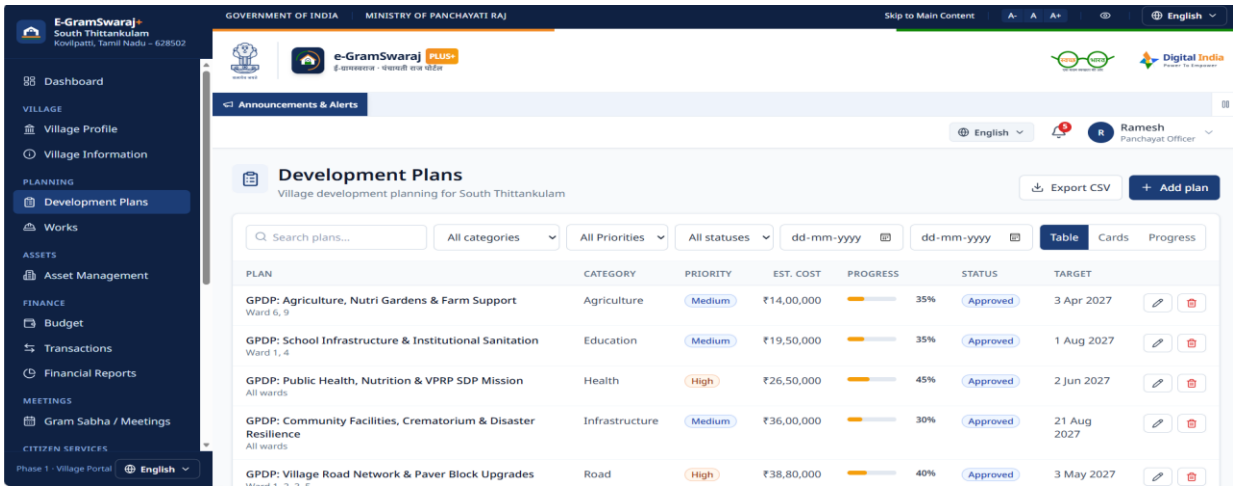


Fig 8.10 Gram Panchayat Development Plan (GPDP) listing

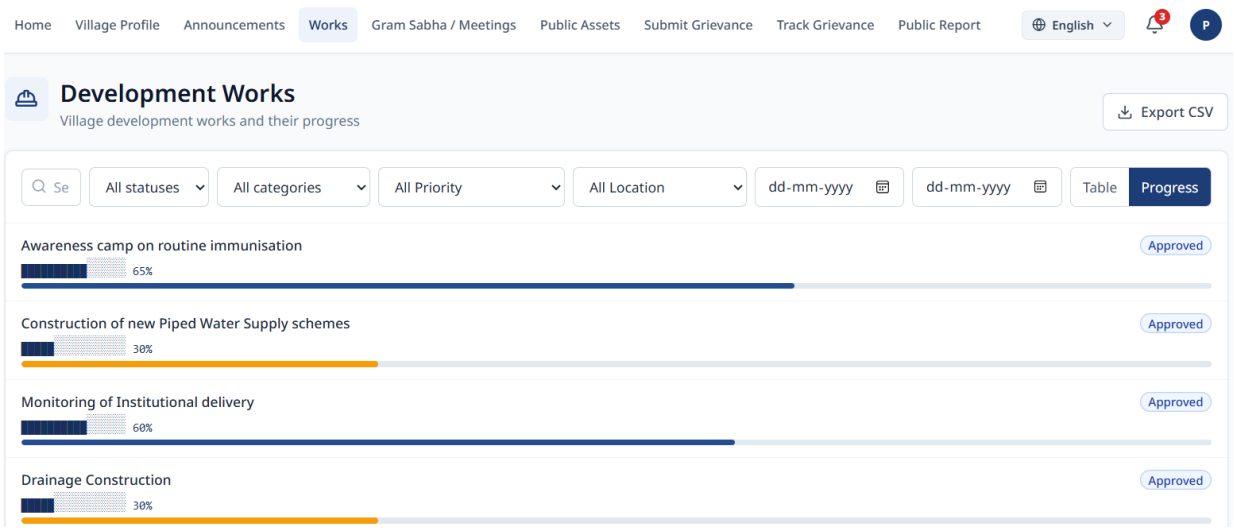


Fig 8.11 Civil works monitoring & progress tracking screen

8.7 PUBLIC ASSET & INFRASTRUCTURE TRACKING

Purpose & Scope

Manages public physical assets across the panchayat (water tanks, hand pumps, street lights, community halls, roads) with GIS coordinates and maintenance schedules.

Functional Requirements

What the module must do:

- Catalog assets by type (WATER_TANK, STREET_LIGHT, RO_PLANT, BUILDING, ROAD), ward location, asset code, and estimated value.
- Store latitude/longitude coordinates to render assets on interactive Leaflet maps.
- Track asset health condition (EXCELLENT, GOOD, POOR, NON_FUNCTIONAL) and log scheduled maintenance dates.

Non-Functional Requirements

How well the module must do it:

- **Geo Validation:** Latitude validated between -90 and 90; longitude validated between -180 and 180.
- **Maintenance Alerts:** Highlights assets whose maintenance interval days have elapsed.

API Design

Method	Endpoint	Purpose / contract
GET	/api/assets	Fetches asset catalog with optional type, ward, and condition filters.
POST	/api/assets	Officer only. Registers a new public asset with GPS location.
PUT	/api/assets/:id	Updates asset health condition or logs maintenance entry.

DB Design

Collection	Keys & indexes	Key fields
assets	_id, villageId, type, condition	name, assetCode, type, ward, latitude, longitude, condition, estimatedValue, maintenanceIntervalDays, status

Screenshots

The screens below show the GIS public asset mapping interface and asset registry.

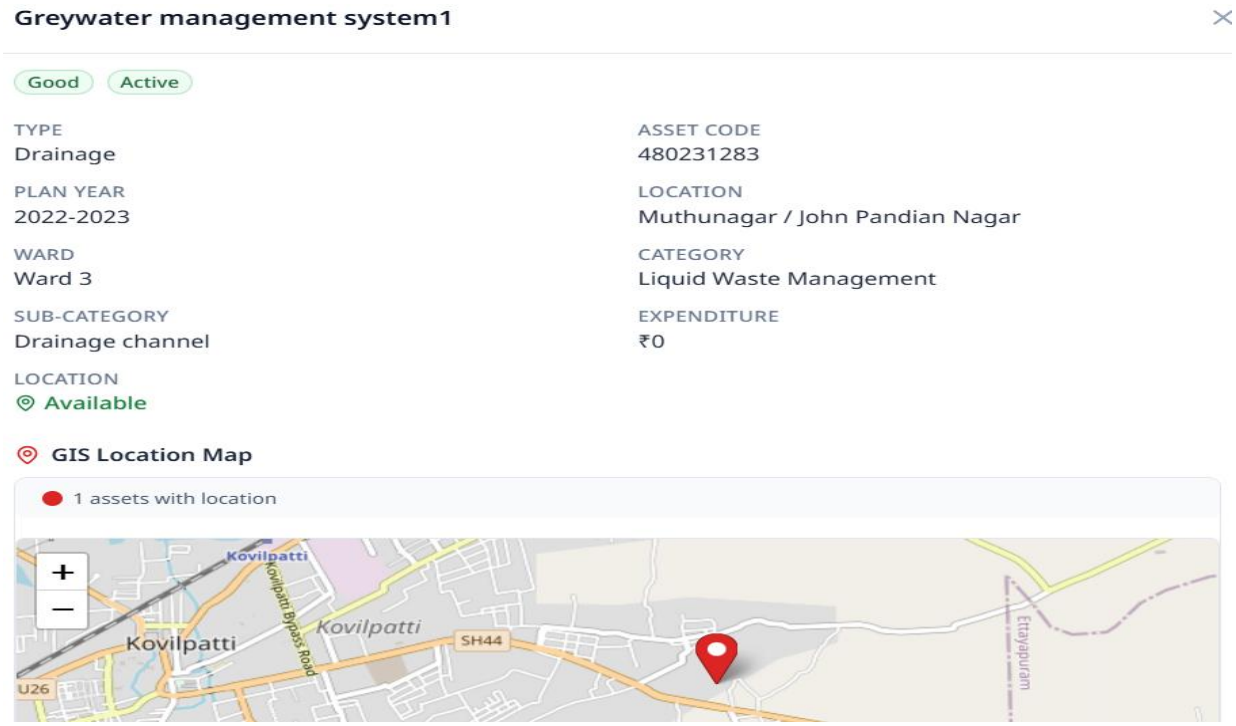


Fig 8.12 Interactive GIS public asset and infrastructure map

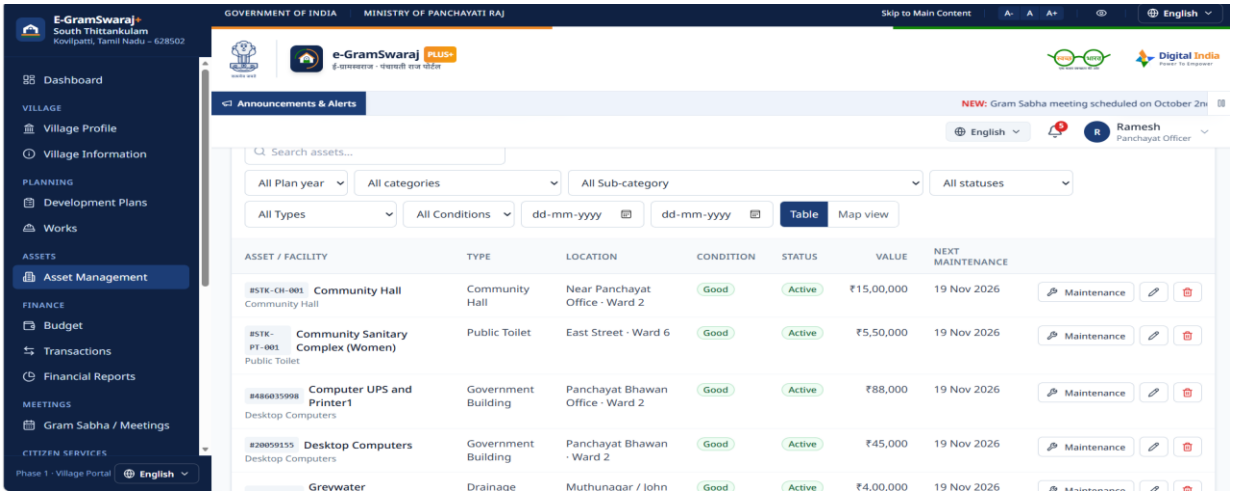


Fig 8.13 Village asset registry and maintenance scheduler

8.8 FINANCIAL TRANSACTIONS & BUDGETARY CONTROL

Purpose & Scope

Enforces strict ledger accounting for village income receipts (government grants, local taxes) and expenditure payments against category-wise annual budgets.

Functional Requirements

What the module must do:

- Record receipts and expenditures with voucher reference numbers and descriptions.
- Define annual category budgets (e.g. 2026-2027) with allocated, utilized, and remaining amounts.
- Sequelize beforeSave hook auto-computes financial year from transaction date and recalculates remaining budget balances.

Non-Functional Requirements

How well the module must do it:

- **Zero Floating Error:** Strict DECIMAL(14,2) representation for all monetary amounts.
- **Budget Guard:** Prevents expenditure booking if category budget utilized amount exceeds allocated funds.

API Design

Method	Endpoint	Purpose / contract
GET	/api/financial-transactions	Fetches financial transactions for a given financial year.
POST	/api/financial-transactions	Officer only. Records a new receipt or expenditure voucher.
GET	/api/budgets	Views category-wise budget allocations and utilization status.

DB Design

Collection	Keys & indexes	Key fields
financialTransactions	_id, villageId, financialYear, type	type, category, amount, referenceNumber, date, financialYear, status
budgets	_id, unique (villageId, financialYear, category)	financialYear, category, allocatedAmount, utilizedAmount, remainingAmount

Screenshots

The screens below show the financial receipts/expenditures ledger and annual budget summary.

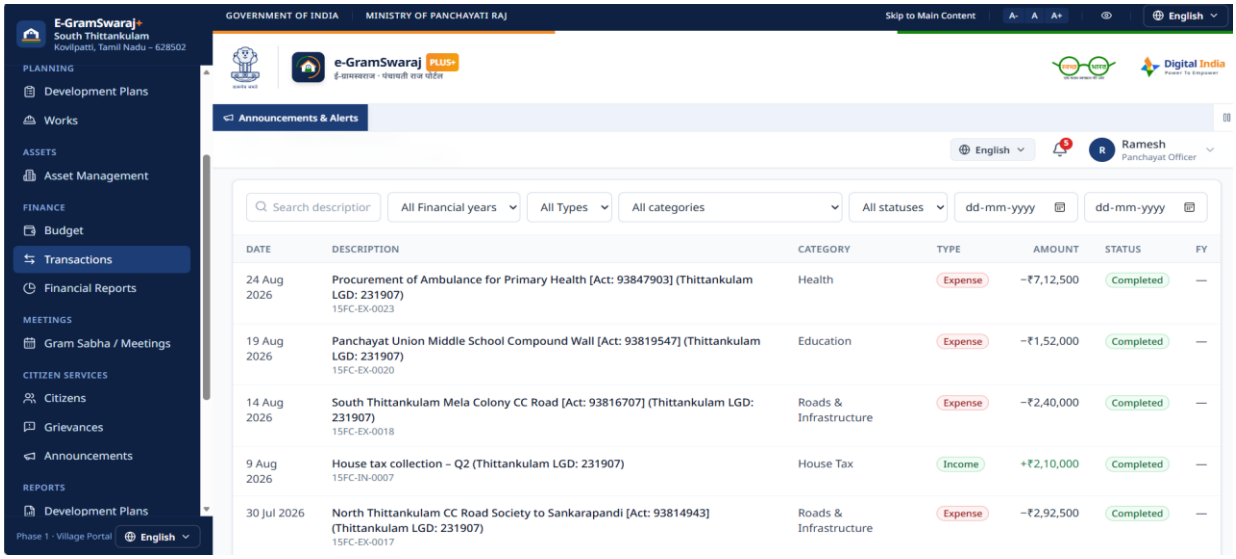


Fig 8.14 Public financial ledger (Receipts & Expenditures)

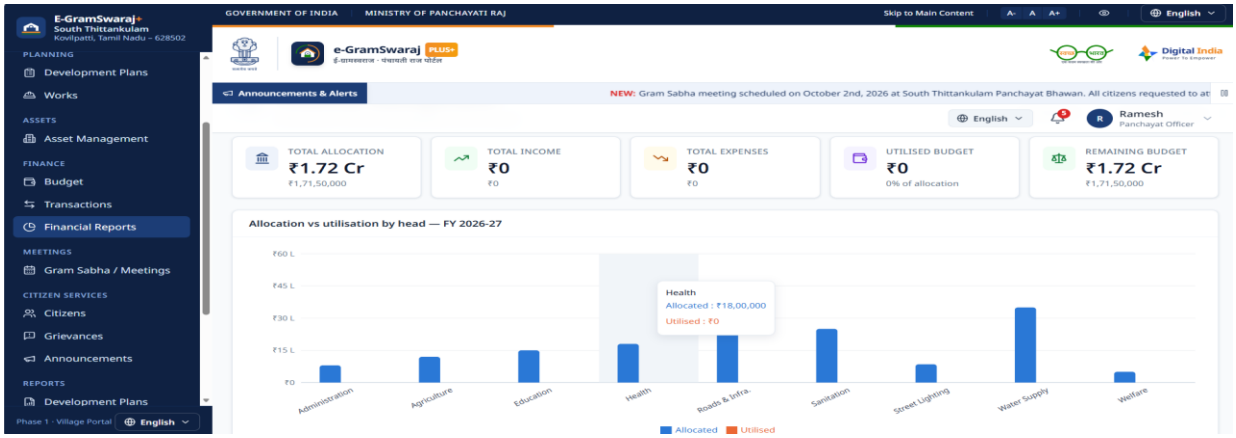


Fig 8.15 Annual budget allocation and utilization charts

8.9 GRAM SABHA MEETINGS & GOVERNANCE

Purpose & Scope

Facilitates democratic local governance by managing public meeting schedules, formal agendas, public attendance logging, and official resolutions.

Functional Requirements

What the module must do:

- Schedule Gram Sabha, Panchayat Board, and Ward Sabha meetings with date, location, and published agenda items.
- Document formal meeting minutes, passed resolutions, and gender-disaggregated public attendance counts (male/female/officials).
- Publish public meeting notices accessible on citizen portals without login requirement.

Non-Functional Requirements

How well the module must do it:

- **Transparency:** Meeting notices marked isPublic=true are publicly readable.
- **Structured Data:** Agenda items and meeting decisions are stored as structured JSON arrays.

API Design

Method	Endpoint	Purpose / contract
GET	/api/meetings	Fetches scheduled or completed meetings for the village.
POST	/api/meetings	Officer only. Schedules new Gram Sabha or Panchayat Board meeting.
PUT	/api/meetings/:id	Uploads meeting minutes, resolutions passed, and public attendance.

DB Design

Collection	Keys & indexes	Key fields
meetings	_id, villageId, type, status	title, type, date, location, agenda (JSON), minutes, decisions (JSON), attendance (JSON)

Screenshots

The screens below show the Gram Sabha meeting notice dashboard and attendance/minutes logger.

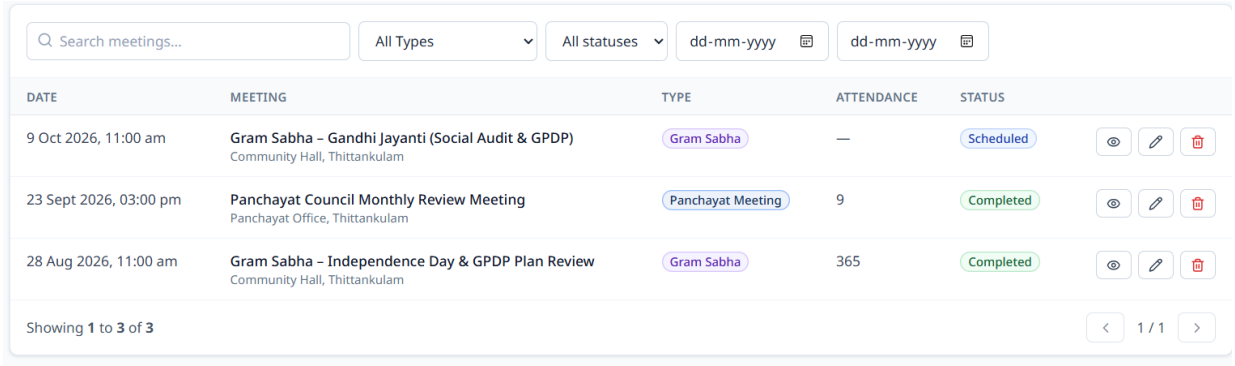


Fig 8.16 Gram Sabha meeting schedule & notice dashboard



Fig 8.17 Gram Sabha meeting resolutions & attendance logger

8.10 EXECUTIVE DASHBOARD, ANALYTICS & LOCALIZATION

Purpose & Scope

Provides operational transparency through live KPI summary monitoring, Recharts analytics graphics, and dynamic 8-language localization powered by Google Cloud Translation API.

Functional Requirements

What the module must do:

- Display active summary counters: total population, total works count, active budget utilization, asset count, and pending grievances.
- Support dynamic UI translation across 8 Indian languages (English, Tamil, Hindi, Telugu, Kannada, Malayalam, Marathi, Bengali).
- Hash source strings (SHA-256) and cache translated text in translationCache to eliminate redundant Google API calls.

Non-Functional Requirements

How well the module must do it:

- **Cache Latency:** < 5 ms for cached translations vs ~300 ms for live API calls.
- **Graceful Fallback:** If translation API key is unconfigured, UI safely displays original source text.

API Design

Method	Endpoint	Purpose / contract
GET	/api/health	Public health check endpoint returning server & database status.
POST	/api/translate	Translates provided text or retrieves from DB cache for target language.

DB Design

Collection	Keys & indexes	Key fields
translationCache	_id, unique sourceHash	sourceHash, sourceText, sourceLanguage, targetLanguage, translatedText

Screenshots

The screens below show the executive analytics dashboard and dynamic multi-lingual interfaces.

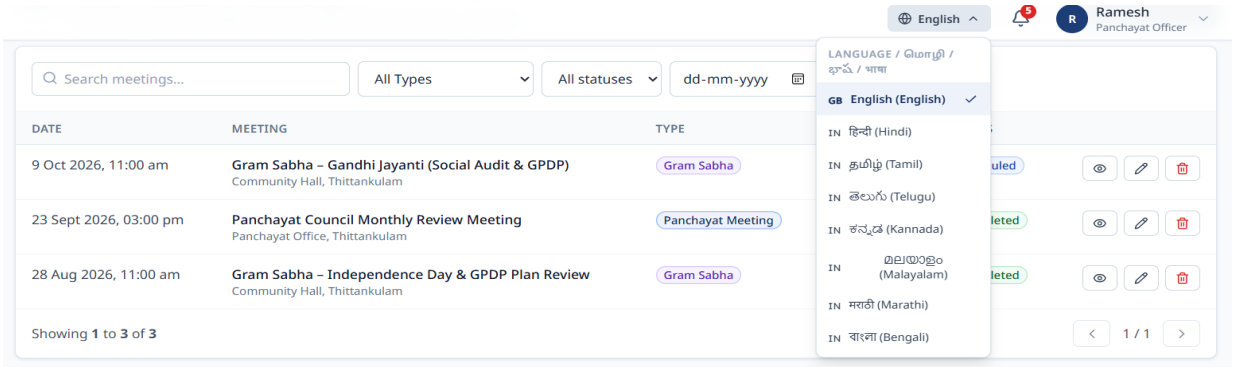


Fig 8.18 Executive analytics dashboard with KPI metrics

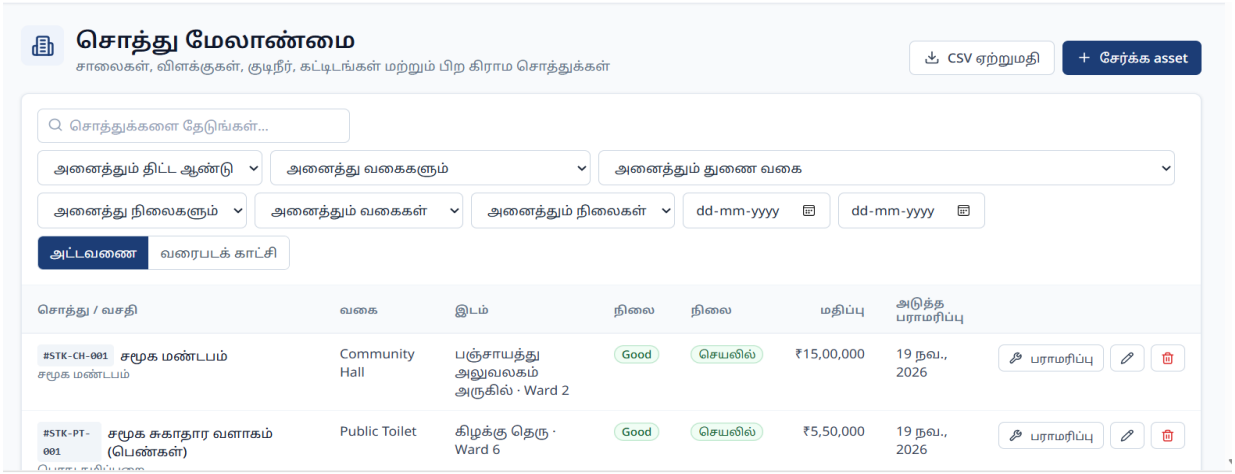


Fig 8.19 Dynamic interface localized in Tamil

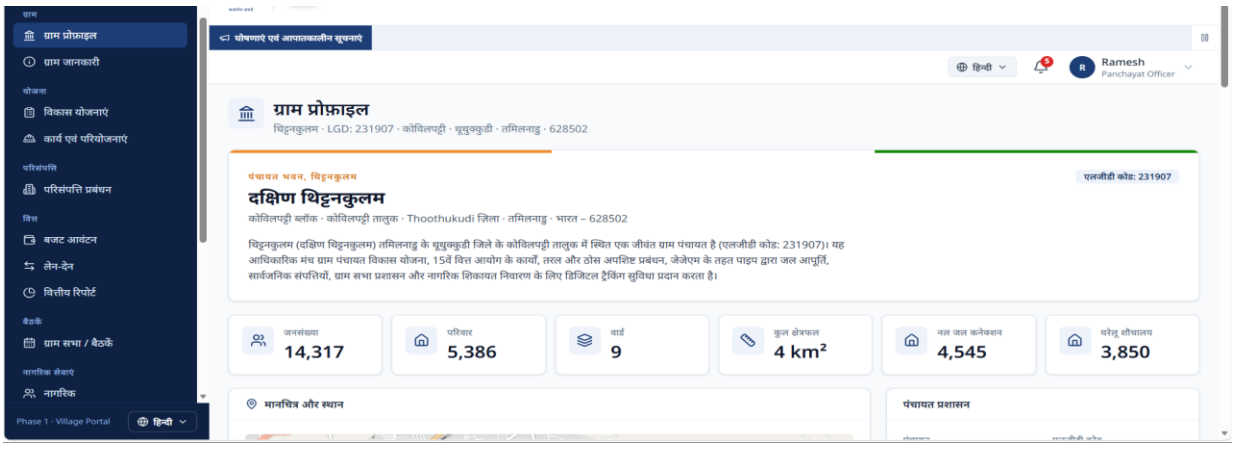


Fig 8.20 Dynamic interface localized in Hindi

CHAPTER 9
CONCLUSION

CONCLUSION

The E-GramSwaraj+ system provides a comprehensive, full-stack digital governance platform for automating and modernizing the core operations of South Thittankulam Village Panchayat. By unifying village profile administration, citizen service requisitions, multi-year development project lifecycle tracking, public physical asset management, financial budget accounting, and Gram Sabha governance within a single web application, the system successfully eliminates manual paper registers, minimizes recording errors, and significantly enhances public transparency.

Developed using React 18, Vite, Tailwind CSS, Recharts, and Leaflet on the client side, and Node.js, Express.js, Sequelize ORM, and MySQL 8 on the server side, the platform exhibits high architectural performance, modular maintainability, and enterprise-grade security. Secured through JWT token authentication, salted bcrypt password hashing, HTTP hardening via Helmet, CORS restrictions, and express rate limiting, the system guarantees role-based access control across administrative officers, staff, and citizens.

Furthermore, the integration of dynamic 8-language localization via Google Cloud Translation API backed by a high-speed SHA-256 database cache breaks down language barriers, enabling local villagers to interact seamlessly with their local panchayat portal in their native language. In conclusion, E-GramSwaraj+ demonstrates how modern web technologies can empower rural digital governance, providing a scalable model capable of extending to Panchayati Raj institutions across the nation.

FUTURE WORK

In future, E-GramSwaraj+ can be improved by integrating real-time SMS notifications for citizens upon grievance status changes and Gram Sabha meeting alerts. The system can be upgraded with automated GIS spatial boundary drawing tools for ward planning, mobile application versions for offline field data collection by village overseers, and AI-powered grievance auto-categorization using natural language processing. Finally, cloud-native deployments with auto-scaling database read replicas can enable multi-panchayat scaling across entire blocks and districts.